Analysis of Algorithm (CSN13400)
ASSIGNMENT-5

Name : Aayansh Niranjan Date :25/08/2026
Reg_No.: 20253004

Q1. Implementation and Analysis of Quick Sort in C: Worst and Average Case Performance Using GNUplot.

A1. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition(int a[], int low, int high){
    int pivot=a[high];
    int i=low-1;
    for(int j=low;j<high;j++){
        if(a[j]<pivot){
            i++;
            int temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }
    int temp=a[i+1];
    a[i+1]=a[high];
    a[high]=temp;
    return i+1;
}

void quickSort(int a[], int low, int high){
    if(low<high){
        int pi=partition(a,low,high);
        quickSort(a,low,pi-1);
        quickSort(a,pi+1,high);
    }
}

double getTime(int a[], int n){
    clock_t start,end;
    start=clock();
    quickSort(a,0,n-1);
    end=clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}
```

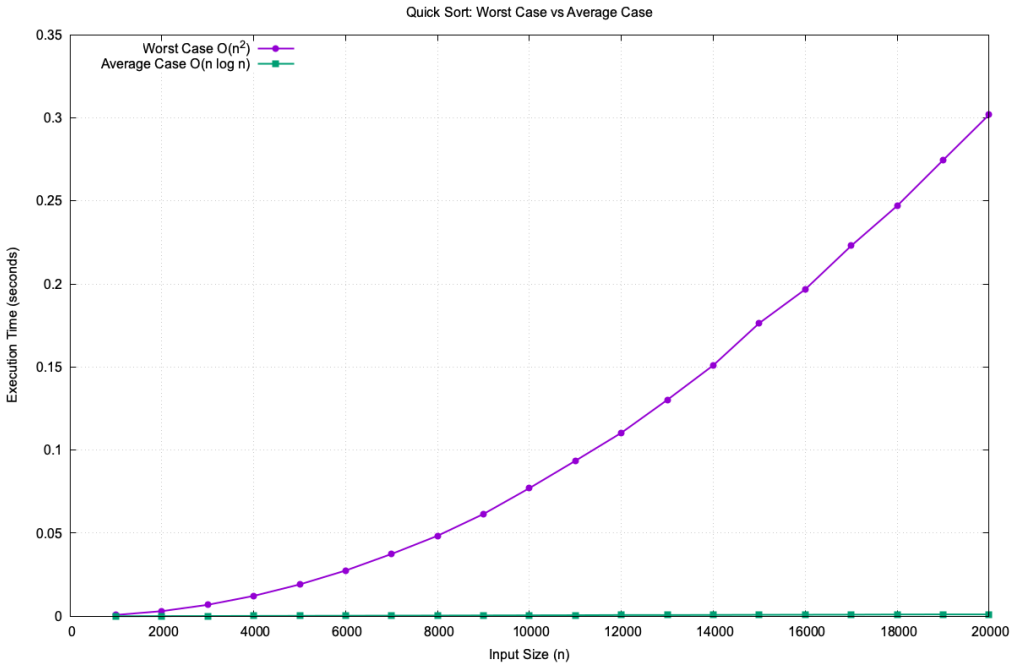
```
int main(){
    FILE *f=fopen("quick.dat","w");
    srand(time(NULL));
    fprintf(f,"# n worst average\n");
    for(int n=1000;n<=20000;n+=1000){
        int worst[n],avg[n];
        for(int i=0;i<n;i++){
            worst[i]=i;
            avg[i]=rand();
        }
        fprintf(f,"%d %lf %lf\n",n,getTime(worst,n),getTime(avg,n));
    }
    fclose(f);
    return 0;
}
```

OUTPUT:

DATA.TXT

```
# n worst average
1000 0.000777 0.000044
2000 0.003069 0.000092
3000 0.006888 0.000142
4000 0.012251 0.000197
5000 0.019149 0.000251
6000 0.027520 0.000310
7000 0.037494 0.000380
8000 0.048437 0.000423
9000 0.061548 0.000493
10000 0.077130 0.000563
11000 0.093586 0.000621
12000 0.110414 0.000730
13000 0.130240 0.000710
14000 0.151044 0.000784
15000 0.176420 0.000860
16000 0.196735 0.000909
17000 0.222934 0.000956
18000 0.246989 0.001051
19000 0.274508 0.001110
20000 0.301821 0.001097
```

GRAPH



Q2. Implementation and Comparative Analysis of Merge Sort and Quick Sort in C for the Average Case Using GNUplot.

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[],int l,int m,int r){
    int n1=m-l+1;
    int n2=r-m;
    int left[n1],right[n2];
    for(int i=0;i<n1;i++){left[i]=a[l+i];}
    for(int i=0;i<n2;i++){right[i]=a[m+1+i];}
    int i=0,j=0,k=l;
    while(i<n1 && j<n2){
        if(left[i]<=right[j]){a[k++]=left[i++];}
        else{a[k++]=right[j++];}
    }
    while(i<n1){a[k++]=left[i++];}
    while(j<n2){a[k++]=right[j++];}
}

void mergeSort(int a[],int l,int r){
    if(l>=r){return;}
    int m=l+(r-l)/2;
    mergeSort(a,l,m);
    mergeSort(a,m+1,r);
    merge(a,l,m,r);
}

int partition(int a[],int low,int high){
    int pivot=a[high];
    int i=low-1;
    for(int j=low;j<high;j++){
        if(a[j]<pivot){
            i++;
            int temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }
    int temp=a[i+1];
    a[i+1]=a[high];
    a[high]=temp;
    return i+1;
}

void quickSort(int a[],int low,int high){
    if(low<high){
        int pi=partition(a,low,high);
        quickSort(a,low,pi-1);
        quickSort(a,pi+1,high);
    }
}
```

```
double getMergeTime(int a[],int n){
   clock_t start,end;
   start=clock();
   mergeSort(a,0,n-1);
   end=clock();
   return (double)(end-start)/CLOCKS_PER_SEC;
}

double getQuickTime(int a[],int n){
   clock_t start,end;
   start=clock();
   quickSort(a,0,n-1);
   end=clock();
   return (double)(end-start)/CLOCKS_PER_SEC;
}

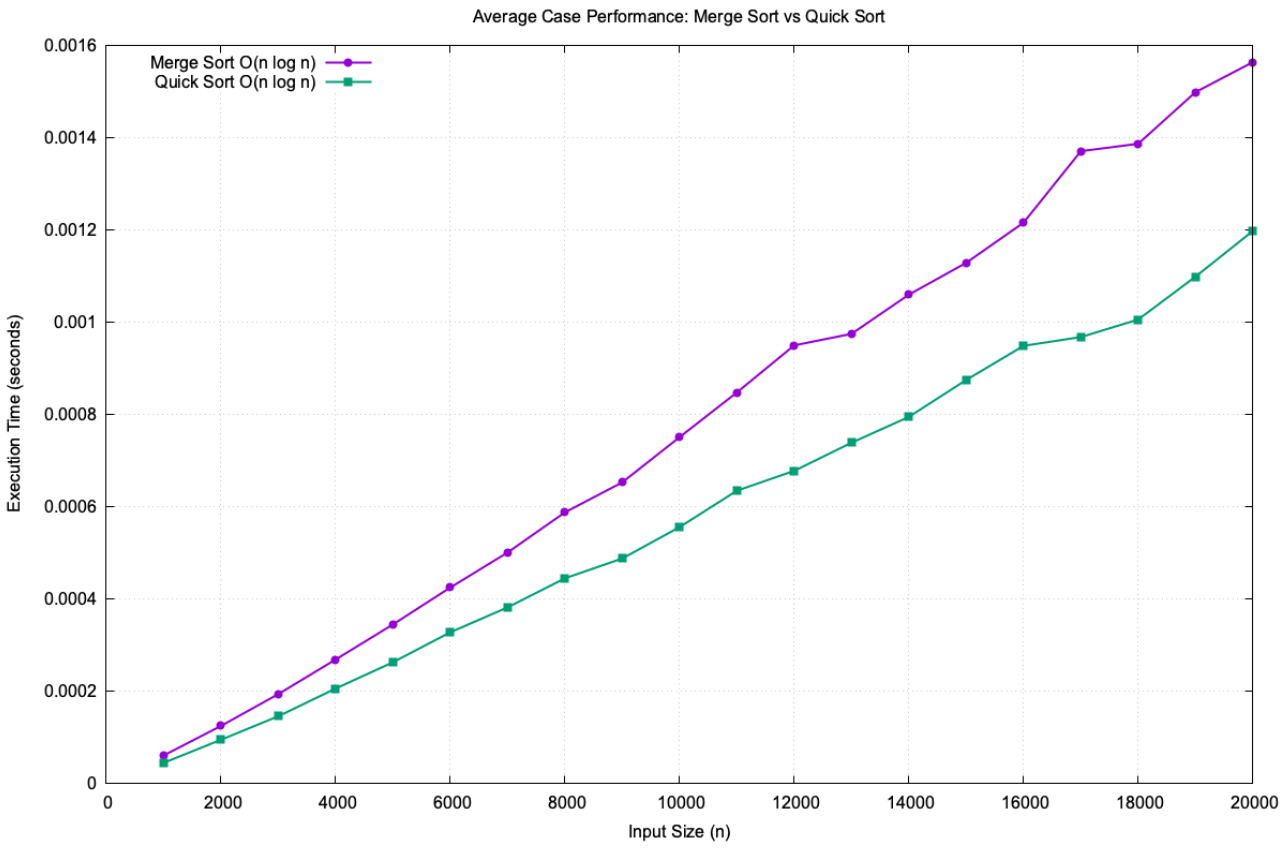
int main(){
   FILE *f=fopen("compare.dat","w");
   srand(time(NULL));
   fprintf(f,"# n merge quick\n");
   for(int n=1000;n<=20000;n+=1000){
      int mergeArray[n];
      int quickArray[n];
      for(int i=0;i<n;i++){
         int x=rand();
         mergeArray[i]=x;
         quickArray[i]=x;
      }
      double mergeTime=getMergeTime(mergeArray,n);
      double quickTime=getQuickTime(quickArray,n);
      fprintf(f,"%d %lf %lf\n",n,mergeTime,quickTime);
   }
   fclose(f);
   return 0;
}
```

OUTPUT:

DATA.TXT

```
# n merge quick
1000 0.000060 0.000044
2000 0.000124 0.000094
3000 0.000193 0.000145
4000 0.000268 0.000205
5000 0.000344 0.000262
6000 0.000424 0.000327
7000 0.000500 0.000381
8000 0.000587 0.000444
9000 0.000652 0.000487
10000 0.000750 0.000555
11000 0.000847 0.000634
12000 0.000949 0.000677
13000 0.000974 0.000738
14000 0.001059 0.000794
15000 0.001128 0.000874
16000 0.001215 0.000948
17000 0.001370 0.000967
18000 0.001386 0.001005
19000 0.001498 0.001098
20000 0.001563 0.001197
```

GRAPH



Q3.Implementation and Analysis of Quick Sort versus Randomised Quick Sort in C for the Worst-Case Analysis Using GNU Plot.

A3. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition(int a[],int low,int high){
    int pivot=a[high];
    int i=low-1;
    for(int j=low;j<high;j++){
        if(a[j]<pivot){
            i++;
            int temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }
    int temp=a[i+1];
    a[i+1]=a[high];
    a[high]=temp;
    return i+1;
}

void quickSort(int a[],int low,int high){
    if(low<high){
        int pi=partition(a,low,high);
        quickSort(a,low,pi-1);
        quickSort(a,pi+1,high);
    }
}

int randomizedPartition(int a[],int low,int high){
    int random=low+rand()%(high-low+1);
    int temp=a[random];
    a[random]=a[high];
    a[high]=temp;
    return partition(a,low,high);
}

void randomizedQuickSort(int a[],int low,int high){
    if(low<high){
        int pi=randomizedPartition(a,low,high);
        randomizedQuickSort(a,low,pi-1);
        randomizedQuickSort(a,pi+1,high);
    }
}
```

```
double getQuickTime(int a[],int n){
    clock_t start,end;
    start=clock();
    quickSort(a,0,n-1);
    end=clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}

double getRandomizedQuickTime(int a[],int n){
    clock_t start,end;
    start=clock();
    randomizedQuickSort(a,0,n-1);
    end=clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}

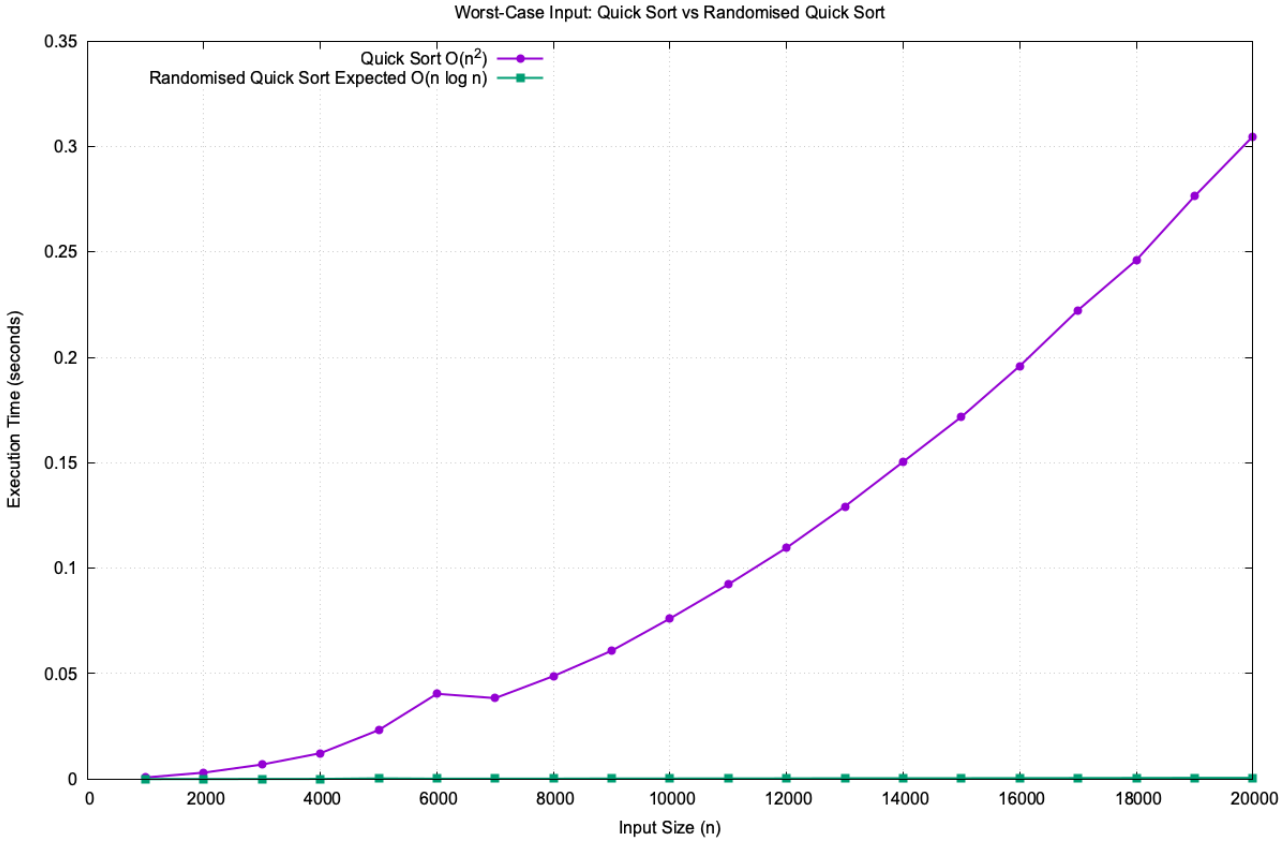
int main(){
    FILE *f=fopen("quick_compare.dat","w");
    srand(time(NULL));
    fprintf(f,"# n quick randomizedQuick\n");
    for(int n=1000;n<=20000;n+=1000){
        int quickArray[n];
        int randomizedArray[n];
        for(int i=0;i<n;i++){
            quickArray[i]=i;
            randomizedArray[i]=i;
        }
        double quickTime=getQuickTime(quickArray,n);
        double randomizedTime=getRandomizedQuickTime(randomizedArray,n);
        fprintf(f,"%d %lf %lf\n",n,quickTime,randomizedTime);
    }
    fclose(f);
    return 0;
}
```

OUTPUT:

DATA.TXT

```
# n quick randomizedQuick
1000 0.000778 0.000028
2000 0.003071 0.000052
3000 0.006922 0.000079
4000 0.012284 0.000106
5000 0.023249 0.000356
6000 0.040445 0.000203
7000 0.038401 0.000207
8000 0.048857 0.000225
9000 0.060912 0.000301
10000 0.076177 0.000302
11000 0.092357 0.000316
12000 0.109720 0.000347
13000 0.129307 0.000409
14000 0.150481 0.000425
15000 0.171798 0.000432
16000 0.195894 0.000477
17000 0.222374 0.000513
18000 0.246104 0.000553
19000 0.276404 0.000592
20000 0.304796 0.000589
```

GRAPH



Q4. Implementation and analysis of quick sort versus randomised quick sort in C for the best and average case analysis using GNU Plot.

A4. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition(int a[],int low,int high){
    int pivot=a[high];
    int i=low-1;
    for(int j=low;j<high;j++){
        if(a[j]<pivot){
            i++;
            int temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }
    int temp=a[i+1];
    a[i+1]=a[high];
    a[high]=temp;
    return i+1;
}

void quickSort(int a[],int low,int high){
    if(low<high){
        int pi=partition(a,low,high);
        quickSort(a,low,pi-1);
        quickSort(a,pi+1,high);
    }
}

int randomizedPartition(int a[],int low,int high){
    int random=low+rand()%(high-low+1);
    int temp=a[random];
    a[random]=a[high];
    a[high]=temp;
    return partition(a,low,high);
}

void randomizedQuickSort(int a[],int low,int high){
    if(low<high){
        int pi=randomizedPartition(a,low,high);
        randomizedQuickSort(a,low,pi-1);
        randomizedQuickSort(a,pi+1,high);
    }
}
```

```
double getQuickTime(int a[],int n){
    clock_t start,end;
    start=clock();
    quickSort(a,0,n-1);
    end=clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}

double getRandomizedQuickTime(int a[],int n){
    clock_t start,end;
    start=clock();
    randomizedQuickSort(a,0,n-1);
    end=clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}

int main(){
    FILE *f=fopen("quick_compare.dat","w");
    srand(time(NULL));
    fprintf(f,"# n quick randomizedQuick\n");
    for(int n=1000;n<=20000;n+=1000){
        int quickArray[n];
        int randomizedArray[n];
        for(int i=0;i<n;i++){
            quickArray[i]=i;
            randomizedArray[i]=i;
        }
        double quickTime=getQuickTime(quickArray,n);
        double randomizedTime=getRandomizedQuickTime(randomizedArray,n);
        fprintf(f,"%d %lf %lf\n",n,quickTime,randomizedTime);
    }
    fclose(f);
    return 0;
}
```

OUTPUT:

DATA.TXT

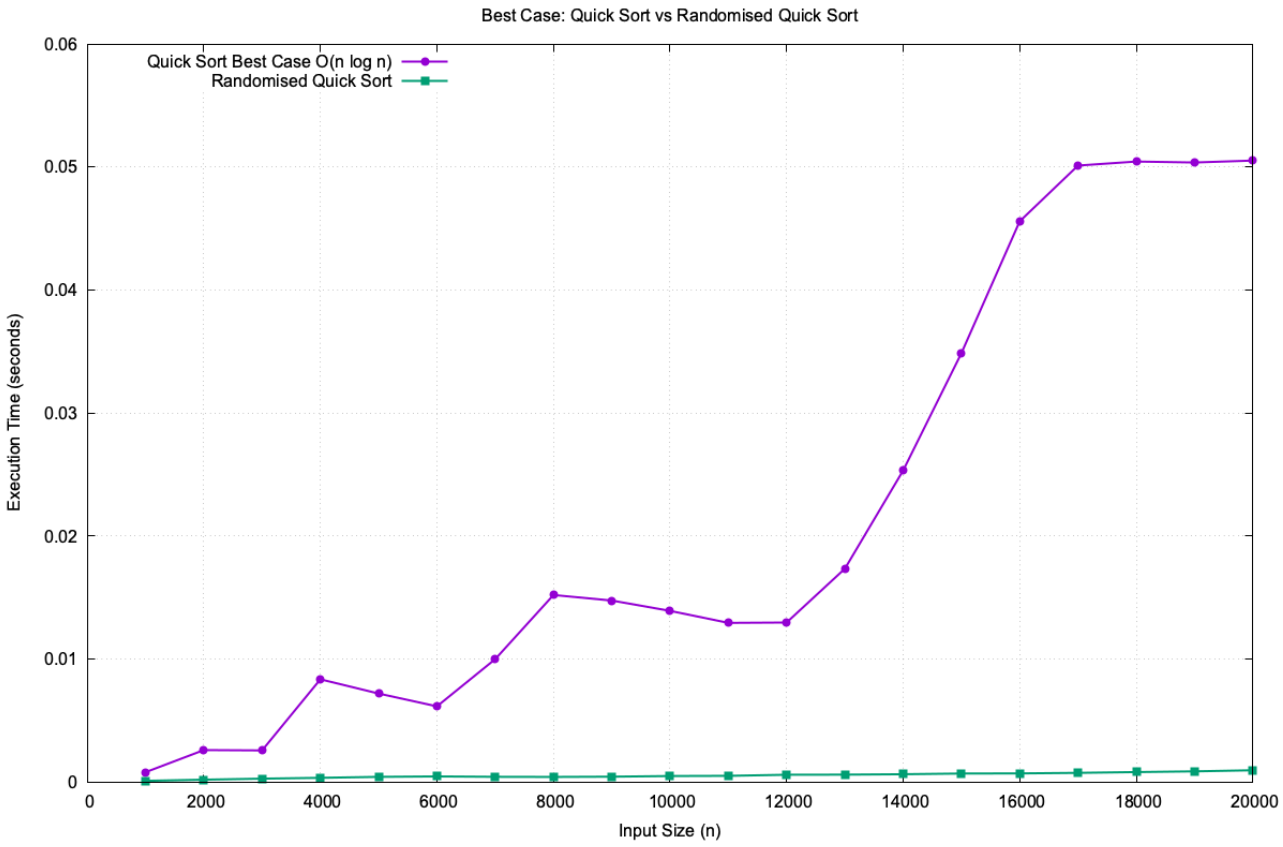
Best

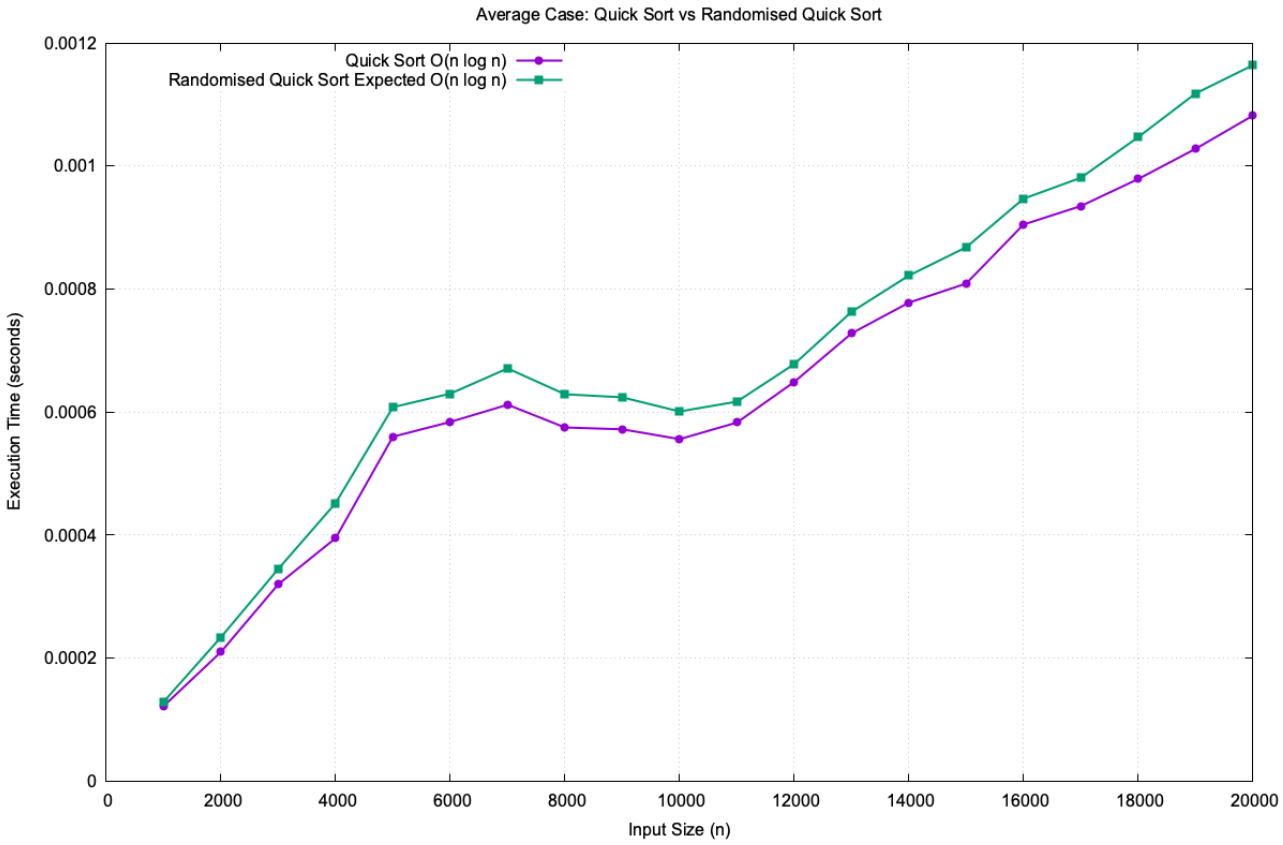
```
# n quick randomized
1000 0.000806 0.000104
2000 0.002605 0.000198
3000 0.002577 0.000281
4000 0.008356 0.000349
5000 0.007199 0.000427
6000 0.006158 0.000462
7000 0.009997 0.000431
8000 0.015211 0.000421
9000 0.014759 0.000441
10000 0.013920 0.000495
11000 0.012945 0.000510
12000 0.012974 0.000598
13000 0.017351 0.000607
14000 0.025335 0.000641
15000 0.034875 0.000698
16000 0.045592 0.000708
17000 0.050123 0.000752
18000 0.050448 0.000816
19000 0.050371 0.000875
20000 0.050527 0.000963
```

Average

```
# n quick randomized
1000 0.000122 0.000129
2000 0.000210 0.000234
3000 0.000320 0.000345
4000 0.000395 0.000452
5000 0.000560 0.000608
6000 0.000584 0.000630
7000 0.000612 0.000671
8000 0.000575 0.000629
9000 0.000572 0.000624
10000 0.000556 0.000601
11000 0.000583 0.000617
12000 0.000649 0.000678
13000 0.000728 0.000763
14000 0.000778 0.000822
15000 0.000809 0.000868
16000 0.000905 0.000947
17000 0.000935 0.000981
18000 0.000979 0.001047
19000 0.001028 0.001118
20000 0.001082 0.001164
```

GRAPH




Average Case: Quick Sort vs Randomised Quick Sort
Quick Sort O(n log n)
Randomised Quick Sort Expected O(n log n)
Execution Time (seconds)
Input Size (n)
0.0012
0.001
0.0008
0.0006
0.0004
0.0002
0
0
2000
4000
6000
8000
10000
12000
14000
16000
18000
20000